

Comparison between various architecture types(A) Stack Architecture

- * Instructions like ADD, SUB, MUL do not require any memory addresses or register names.
- * These instructions are called 0-Address instructions. because the CPU automatically assumes it needs to operate on whatever two numbers are on Top of Stack (Tos).

PUSH X : Finds the data from the RAM and place it on top of the stack.

POP Z : Take the top number off the stack and saves it into the RAM.

Ex:- $Z = X + Y$

PUSH X

PUSH Y

ADD $\rightarrow X + Y$

POP Z $\rightarrow Z = (X + Y)$

Ex:- $Y = A/B - (A - C * B)$

PUSH A

PUSH B

DIV $\rightarrow A/B$

PUSH A

PUSH C

PUSH B

MUL $\rightarrow C * B$

SUB $\rightarrow A - (C * B)$

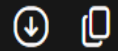
SUB $\rightarrow (A/B) - (A - (C * B))$

POP Y

starts from
* Left to right

The stack processor runs this sequence: PDF

Plaintext



```
PUSH A    // Stack: [A]
PUSH B    // Stack: [B, A]
DIV       // Stack: [ (A/B) ]

PUSH A    // Stack: [A, (A/B)]
PUSH C    // Stack: [C, A, (A/B)]
PUSH B    // Stack: [B, C, A, (A/B)]
MUL       // Multiplies C*B -> Stack: [ (C*B), A, (A/B) ]
SUB       // Subtracts (A - C*B) -> Stack: [ (A - C*B), (A/B) ]
SUB       // Subtracts (A/B) - (A - C*B) -> Stack: [ Final Result ]

POP Y     // Saves Final Result to RAM location Y
```


(B) Accumulator Architecture

- * Used in older computers
- * Can hold one number when executing.

LOAD X : Copies data from the RAM to the Accumulator
 $(ACC = Mem[X])$

STORE Y : Copies the current value in the Accumulator
 to the RAM $(Mem[Y] = ACC)$

Ex:- $Z = X + Y$

LOAD X

ADD Y $\rightarrow X + Y$

STORE Z $\rightarrow Z = (X + Y)$

Ex:- $Y = A/B - (A - C \times B)$

LOAD C

MUL B $\rightarrow C \times B$

STORE D $\rightarrow D = (C \times B)$

LOAD A

SUB D $\rightarrow A - D$

STORE D $\rightarrow D = (A - D)$

LOAD A

DIV B $\rightarrow A/B$

SUB D $\rightarrow (A/B) - D$

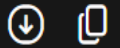
STORE Y $\rightarrow Y = (A/B) - D$

$Y = (A/B) - (A - C \times B)$

Starts from
Right to
left

A temp
location in
the RAM

Plaintext



```
// Part 1: Calculate (C * B)
LOAD C      // ACC = C
MUL B       // ACC = C * B
STORE D     // Save temporary result (C * B) in RAM location D

// Part 2: Calculate (A - D) -> which is (A - C * B)
LOAD A      // ACC = A
SUB D       // ACC = A - D
STORE D     // Save temporary result (A - C * B) in RAM location D

// Part 3: Calculate (A / B) - D
LOAD A      // ACC = A
DIV B       // ACC = A / B
SUB D       // ACC = (A / B) - D
STORE Y     // Save final answer to RAM location Y
```


(c) Memory - Memory architecture.

* This uses 2-Address instruction format as well as 3-Address instruction format.

* 3-Address instructions

Format: OPERAND Destination, SOURCE1, SOURCE2

Ex:-

ADD Z, X, Y

Reads from X and Y
Stores in Z

* 2-Address instructions

Format: OPERAND Destination and SOURCE1, SOURCE2

Ex:-

ADD X, Y

reads from X and Y and overwrites X

Ex:- $Y = A/B - (A - C * B)$

3-operand memory sequence

DIV D, A, B $\rightarrow D = A/B$

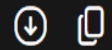
MUL E, C, B $\rightarrow E = C * B$

SUB E, A, E $\rightarrow E = A - E$

SUB Y, D, E $\rightarrow Y = D - E$

3-Operand Memory Sequence

Plaintext



```
DIV D, A, B    // Mem[D] = Mem[A] / Mem[B]
MUL E, C, B    // Mem[E] = Mem[C] * Mem[B]
SUB E, A, E    // Mem[E] = Mem[A] - Mem[E]
SUB Y, D, E    // Mem[Y] = Mem[D] - Mem[E]
```


(D) LOAD - STORE Architecture.

- * This is the standard model used in all modern processors.
- * The design follows one golden rule: Arithmetic & logical calculations are never allowed to touch RAM directly.
- * General Purpose Registers are used.

Ex:- $Z = X + Y$

LOAD R1, X

LOAD R2, Y

ADD R3, R1, R2 $\rightarrow R3 = R1 + R2$

STORE Z, R3

Ex:- $Y = A/B - (A - C \times B)$

LOAD R1, A

LOAD R2, B

LOAD R3, C

DIV R4, R1, R2 $\rightarrow R4 = A/B$

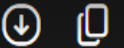
MUL R5, R3, R2 $\rightarrow R5 = C \times B$

SUB R5, R1, R5 $\rightarrow R5 = A - (C \times B)$

SUB R6, R4, R5 $\rightarrow R6 = (A/B) - (A - (C \times B))$

STORE Y, R6

Plaintext



// Step 1: Load all required raw variables from RAM into registers

```
LOAD R1, A      // R1 = Mem[A]
LOAD R2, B      // R2 = Mem[B]
LOAD R3, C      // R3 = Mem[C][cite: 1]
```

// Step 2: Perform all math purely inside registers

```
DIV R4, R1, R2   // R4 = A / B[cite: 1]
MUL R5, R3, R2   // R5 = C * B[cite: 1]
SUB R5, R1, R5    // R5 = A - (C * B)[cite: 1]
SUB R4, R4, R5    // R4 = (A / B) - (A - C * B)[cite: 1]
```

// Step 3: Write the final calculated result back to RAM

```
STORE Y, R4      // Mem[Y] = R4[cite: 1]
```